

AGILE DEVELOPMENT PROCESS AND DEVOPS LAB FAT

Name: Kevin R Abraham
Reg No: 23MIS0061
Faculty: Dr. BALA GANESH N
Date: 15 April 2026

SET 5

Online Donation System

Problem Statement:

Ensure donations are processed and recorded accurately

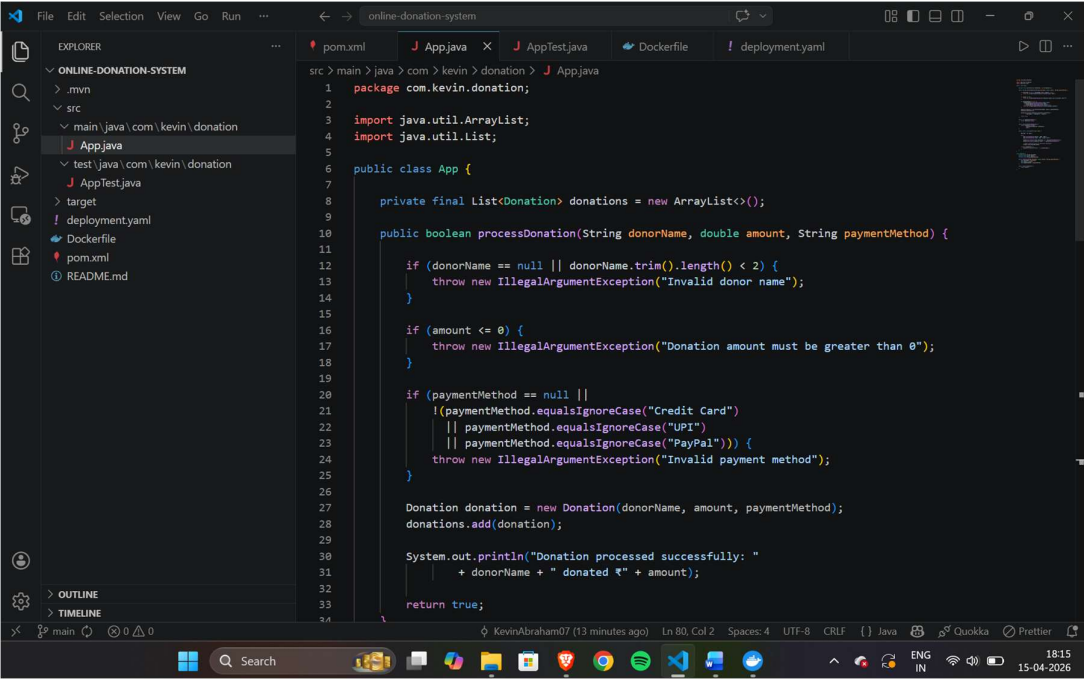
Tasks:

1. Develop donation processing logic
2. Write JUnit tests for transaction validation
3. Implement CI/CD pipeline with automated testing
 - Set up version control using GitHub
 - Configure CI pipeline to trigger on code commits
 - Automate build process using Maven
 - Integrate automated unit tests in pipeline
 - Configure CD pipeline for automatic deployment to staging/production
4. Deploy services on Docker and Kubernetes
 - Containerize application using Docker
 - Build and push Docker image to container registry
 - Create Kubernetes deployment and service YAML files

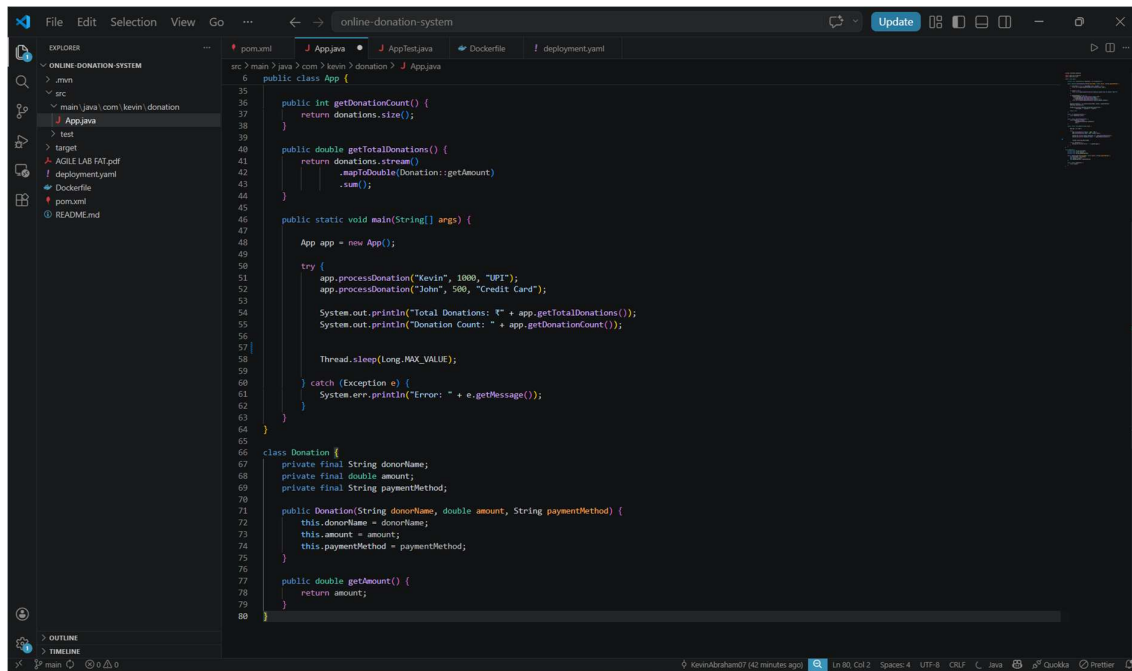
GitHub Repository Link: <https://github.com/KevinAbraham07/Agile-FAT.git>

App.java

Contains Java code for Online Donation System



```
1 package com.kevin.donation;
2
3 import java.util.ArrayList;
4 import java.util.List;
5
6 public class App {
7
8     private final List<Donation> donations = new ArrayList<>();
9
10    public boolean processDonation(String donorName, double amount, String paymentMethod) {
11
12        if (donorName == null || donorName.trim().length() < 2) {
13            throw new IllegalArgumentException("Invalid donor name");
14        }
15
16        if (amount <= 0) {
17            throw new IllegalArgumentException("Donation amount must be greater than 0");
18        }
19
20        if (paymentMethod == null ||
21            !(paymentMethod.equalsIgnoreCase("Credit Card")
22              || paymentMethod.equalsIgnoreCase("UPI")
23              || paymentMethod.equalsIgnoreCase("PayPal"))) {
24            throw new IllegalArgumentException("Invalid payment method");
25        }
26
27        Donation donation = new Donation(donorName, amount, paymentMethod);
28        donations.add(donation);
29
30        System.out.println("Donation processed successfully: "
31                           + donorName + " donated ₹" + amount);
32
33        return true;
34    }
35 }
```

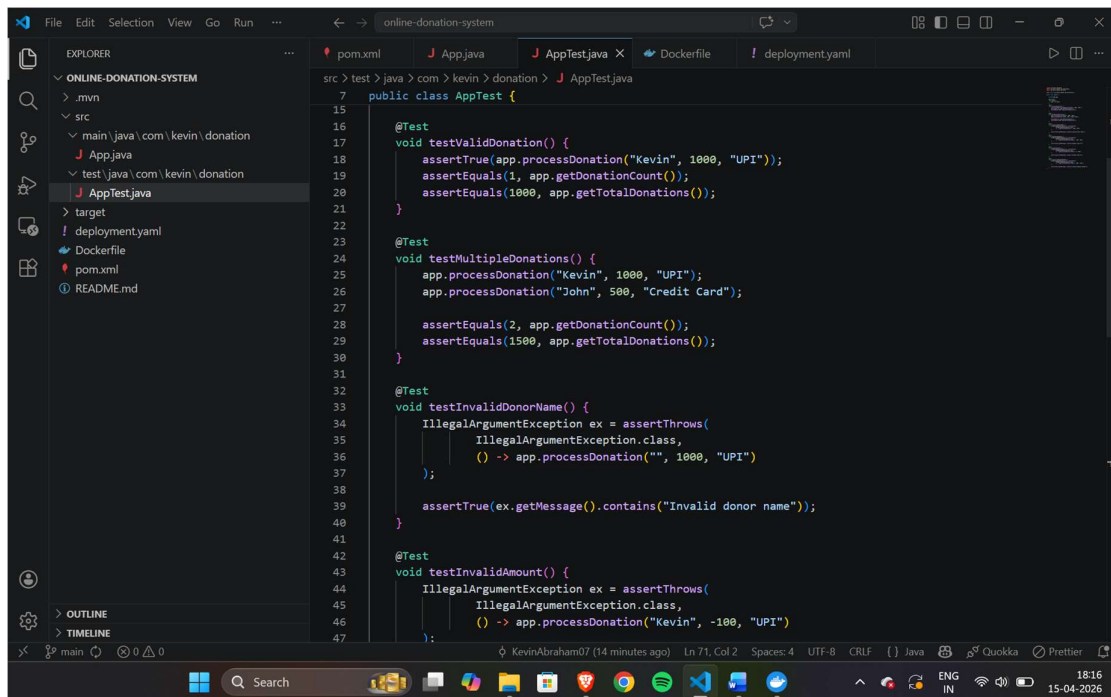


The screenshot shows an IDE with the 'online-donation-system' project open. The 'App.java' file is selected in the Explorer and is being edited. The code defines an 'App' class with methods for getting donation counts and totals, and a 'main' method that processes donations. A 'Donation' class is also defined as a private static inner class.

```
src > main > java > com > kevin > donation > J App.java
6 public class App {
35
36     public int getDonationCount() {
37         return donations.size();
38     }
39
40     public double getTotalDonations() {
41         return donations.stream()
42             .mapToDouble(Donation::getAmount)
43             .sum();
44     }
45
46     public static void main(String[] args) {
47
48         App app = new App();
49
50         try {
51             app.processDonation("Kevin", 1000, "UPI");
52             app.processDonation("John", 500, "Credit Card");
53
54             System.out.println("Total Donations: " + app.getTotalDonations());
55             System.out.println("Donation Count: " + app.getDonationCount());
56
57             Thread.sleep(Long.MAX_VALUE);
58         } catch (Exception e) {
59             System.err.println("Error: " + e.getMessage());
60         }
61     }
62
63 }
64
65 class Donation {
66     private final String donorName;
67     private final double amount;
68     private final String paymentMethod;
69
70     public Donation(String donorName, double amount, String paymentMethod) {
71         this.donorName = donorName;
72         this.amount = amount;
73         this.paymentMethod = paymentMethod;
74     }
75
76     public double getAmount() {
77         return amount;
78     }
79
80 }
```

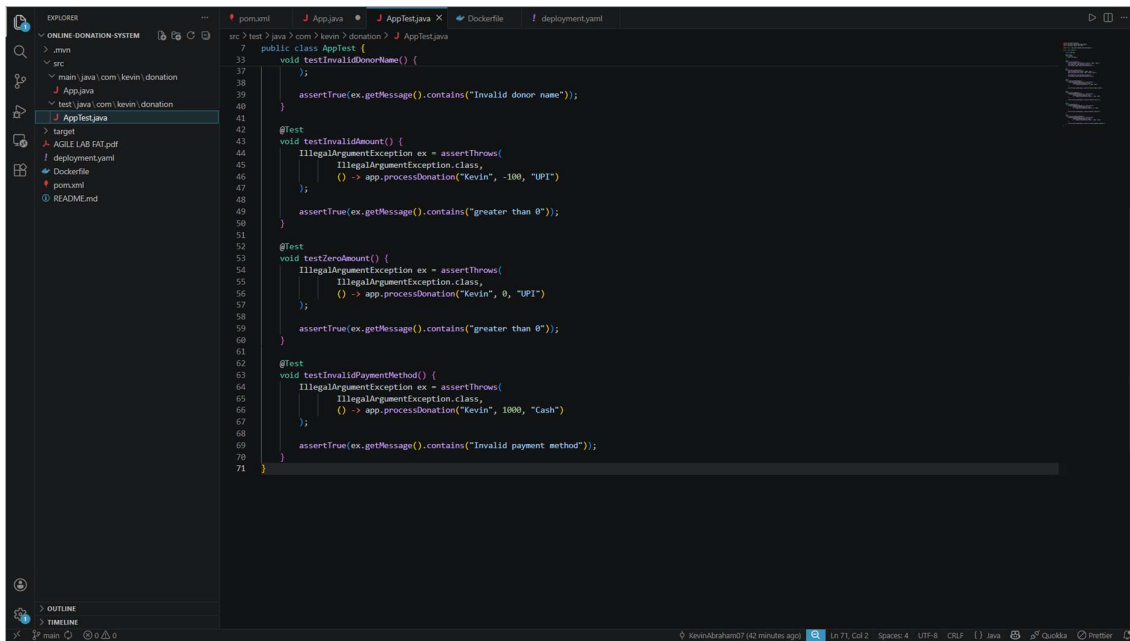
AppTest.java

Contains JUnit Testing Code for Online Donation System



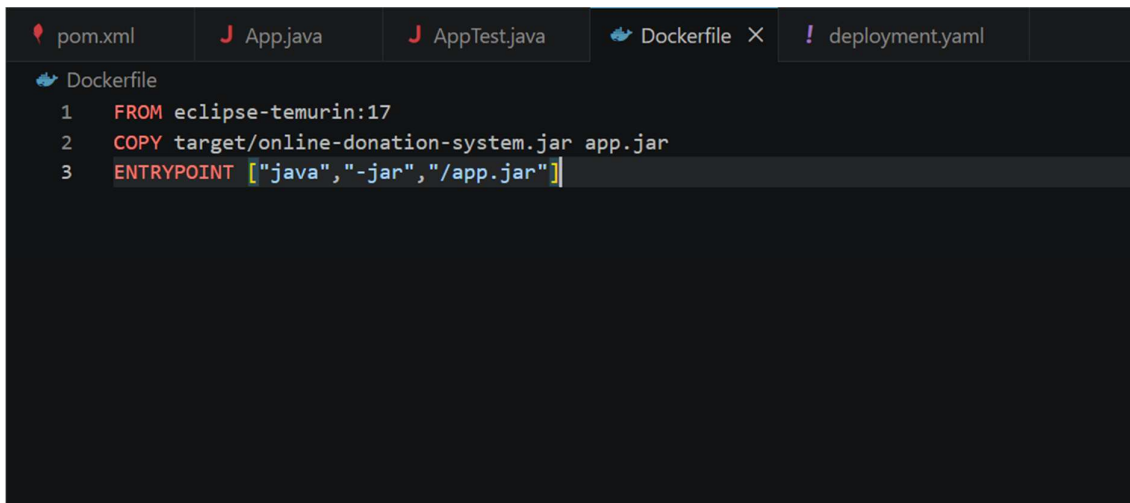
The screenshot shows the same IDE with the 'AppTest.java' file selected. The code contains JUnit tests for the 'App' class, including tests for valid donations, multiple donations, invalid donor names, and invalid amounts.

```
src > test > java > com > kevin > donation > J AppTest.java
7 public class AppTest {
15
16     @Test
17     void testValidDonation() {
18         assertTrue(app.processDonation("Kevin", 1000, "UPI"));
19         assertEquals(1, app.getDonationCount());
20         assertEquals(1000, app.getTotalDonations());
21     }
22
23     @Test
24     void testMultipleDonations() {
25         app.processDonation("Kevin", 1000, "UPI");
26         app.processDonation("John", 500, "Credit Card");
27
28         assertEquals(2, app.getDonationCount());
29         assertEquals(1500, app.getTotalDonations());
30     }
31
32     @Test
33     void testInvalidDonorName() {
34         IllegalArgumentException ex = assertThrows(
35             IllegalArgumentException.class,
36             () -> app.processDonation("", 1000, "UPI")
37         );
38
39         assertTrue(ex.getMessage().contains("Invalid donor name"));
40     }
41
42     @Test
43     void testInvalidAmount() {
44         IllegalArgumentException ex = assertThrows(
45             IllegalArgumentException.class,
46             () -> app.processDonation("Kevin", -100, "UPI")
47         );
48     }
49 }
```



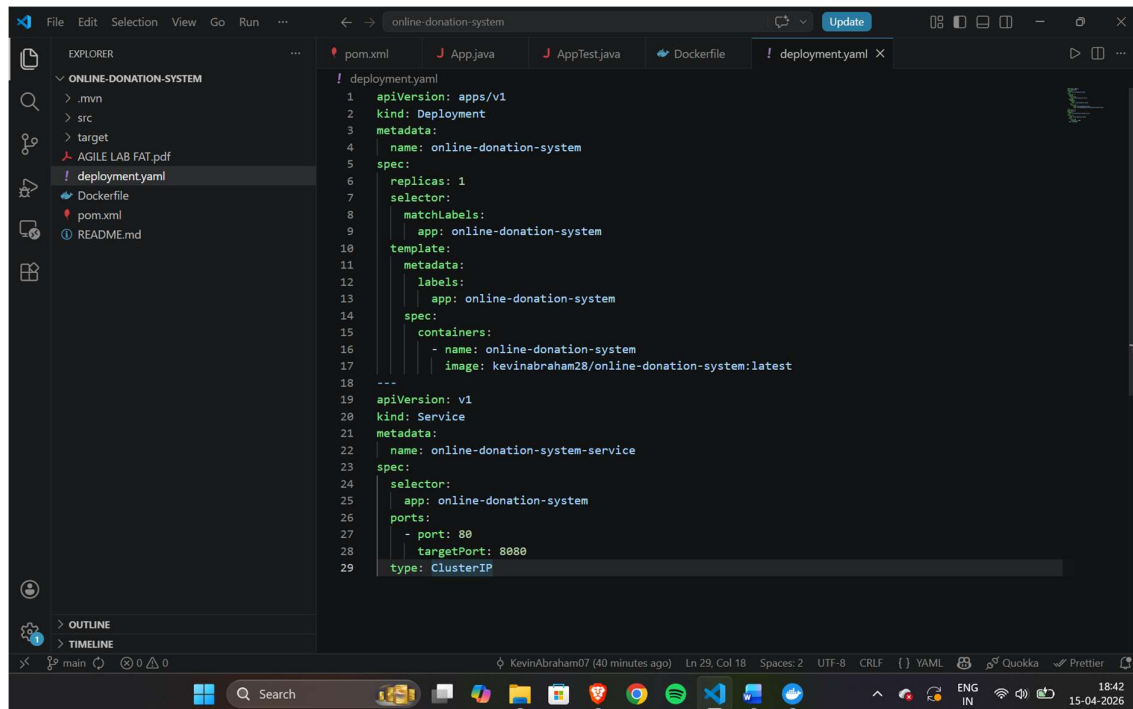
```
src > test > java > com > kevin > donation > AppTest.java
7 public class AppTest {
33 void testInvalidDonorName() {
37     };
38
39     assertTrue(ex.getMessage().contains("Invalid donor name"));
40 }
41
42
43 @Test
44 void testInvalidAmount() {
45     IllegalArgumentException ex = assertThrows(
46         IllegalArgumentException.class,
47         () -> app.processDonation("Kevin", -100, "UPI")
48     );
49     assertTrue(ex.getMessage().contains("greater than 0"));
50 }
51
52 @Test
53 void testZeroAmount() {
54     IllegalArgumentException ex = assertThrows(
55         IllegalArgumentException.class,
56         () -> app.processDonation("Kevin", 0, "UPI")
57     );
58     assertTrue(ex.getMessage().contains("greater than 0"));
59 }
60
61
62 @Test
63 void testInvalidPaymentMethod() {
64     IllegalArgumentException ex = assertThrows(
65         IllegalArgumentException.class,
66         () -> app.processDonation("Kevin", 1000, "Cash")
67     );
68     assertTrue(ex.getMessage().contains("Invalid payment method"));
69 }
70
71 }
```

Dockerfile



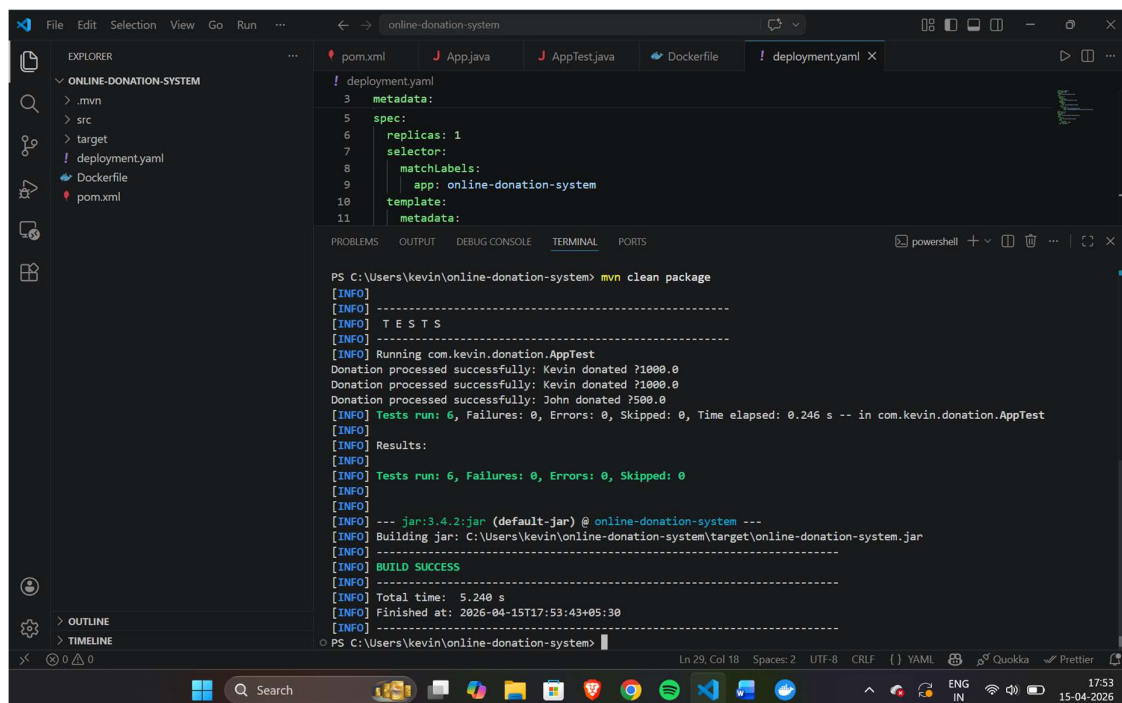
```
pom.xml App.java AppTest.java Dockerfile X deployment.yaml
Dockerfile
1 FROM eclipse-temurin:17
2 COPY target/online-donation-system.jar app.jar
3 ENTRYPOINT ["java", "-jar", "/app.jar"]
```

deployment.yaml



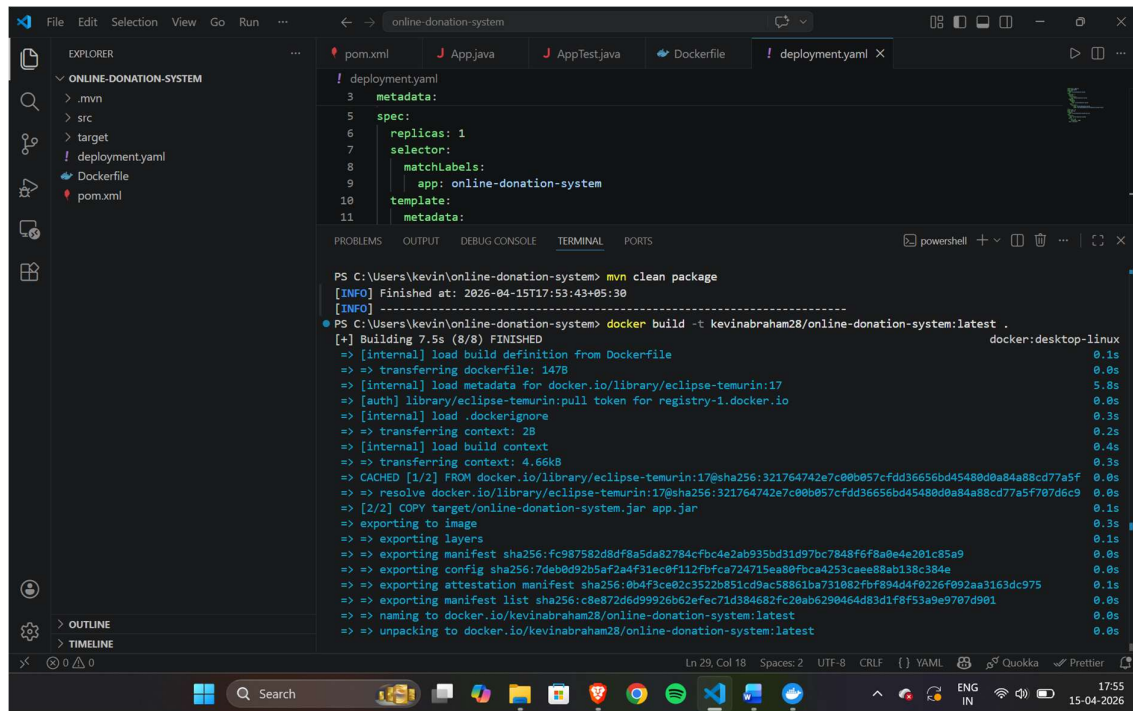
mvn clean package

Compiles the Java application, runs tests, and packages it into an executable JAR file.



docker build -t kevinabraham28/online-donation-system:latest .

Builds a Docker image for the application and tags it for Docker Hub.



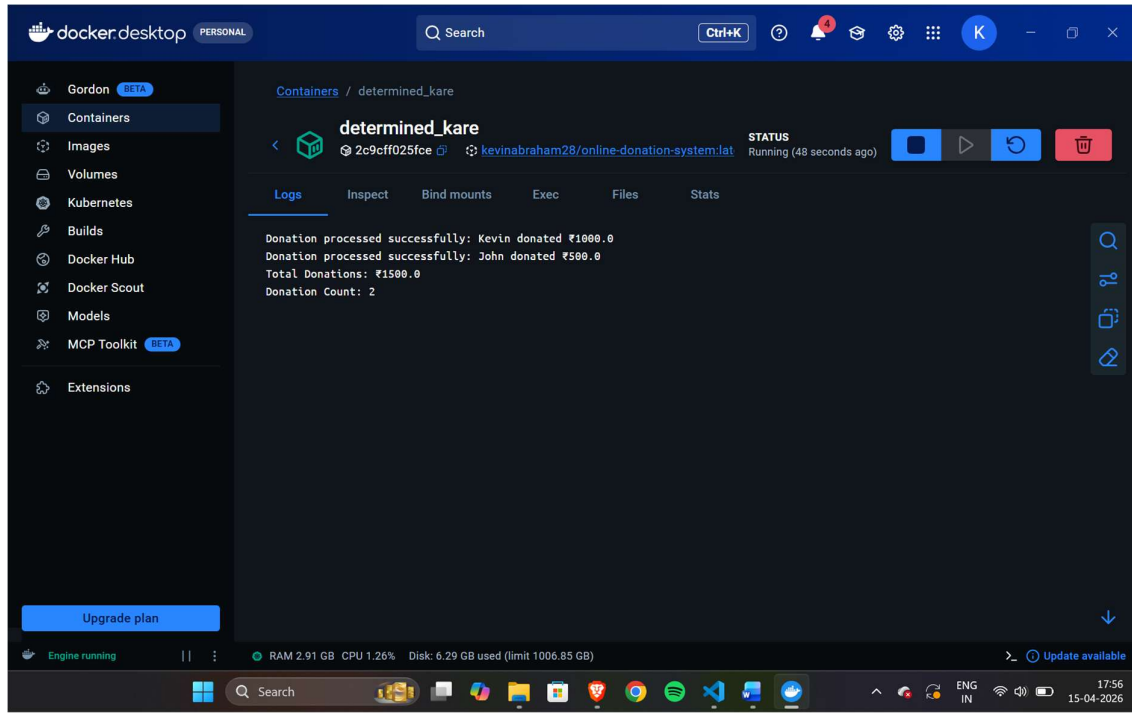
docker run kevinabraham28/online-donation-system:latest

Runs the built Docker image as a container locally for testing.

```
PS C:\Users\kevin\online-donation-system> docker run kevinabraham28/online-donation-system:latest
Donation processed successfully: Kevin donated ₹1000.0
Donation processed successfully: John donated ₹500.0
Total Donations: ₹1500.0
Donation Count: 2
```

New Docker Container Created

A Docker container was successfully created and run from the built image, therefore confirming the creation of cluster



docker push kevinabraham28/event-registration-app:latest

Uploads the Docker image to Docker Hub for remote storage and deployment.

```
PS C:\Users\kevin\online-donation-system> docker push kevinabraham28/online-donation-system:latest
The push refers to repository [docker.io/kevinabraham28/online-donation-system]
a1e6e1a167f1: Mounted from kevinabraham28/todo-management-app
d0061ec7f38e: Pushed
38d0b5bce0f2: Pushed
689b91d88a0f: Mounted from kevinabraham28/todo-management-app
a7428f4f5c1d: Mounted from kevinabraham28/todo-management-app
a149c623348d: Mounted from kevinabraham28/todo-management-app
f00bd8c488bb: Mounted from kevinabraham28/todo-management-app
latest: digest: sha256:c8e872d6d99926b62efec71d384682fc20ab6290464d83d1f8f53a9e9707d901 size: 856
```

kubectl apply -f deployment.yaml

Deploys the application in the Kubernetes cluster using the deployment configuration.

```
PS C:\Users\kevin\online-donation-system> kubectl apply -f deployment.yaml
deployment.apps/online-donation-system created
service/online-donation-system-service created
```

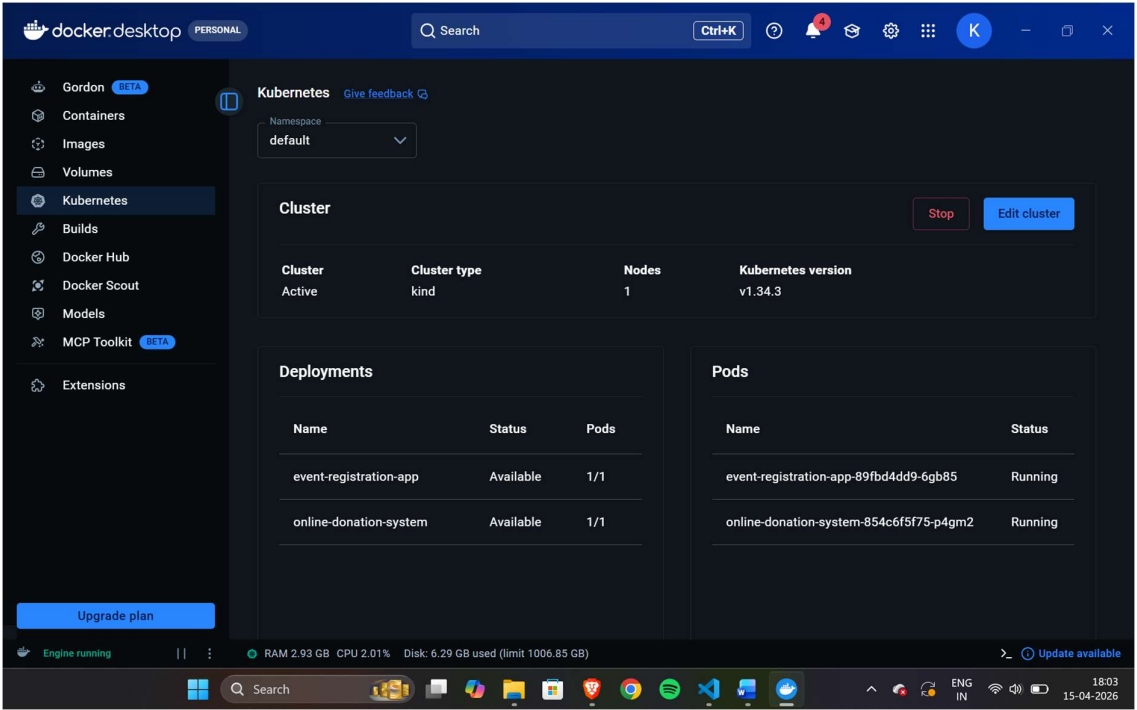

kubectl get pods

Displays the status of all Kubernetes pods in the cluster to verify that the deployed application container is running successfully.

```
PS C:\Users\kevin\online-donation-system> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
event-registration-app-89fbd4dd9-6gb85 1/1     Running   1 (58m ago) 4d19h
online-donation-system-854c6f5f75-p4gm2 1/1     Running   0           22m
```

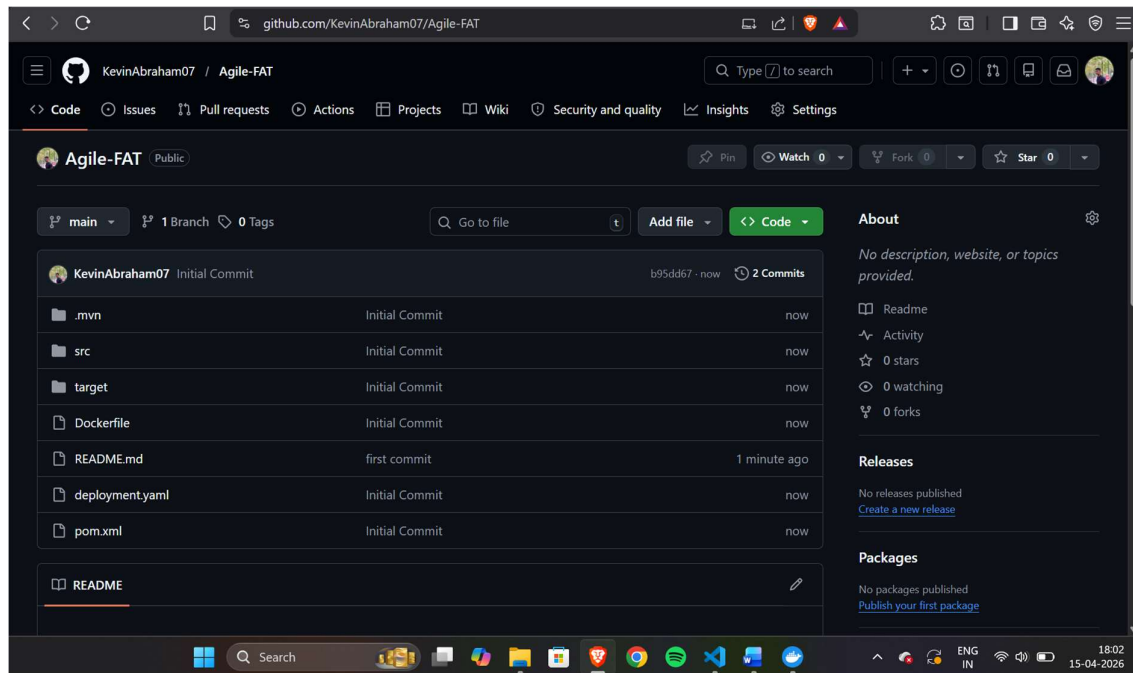
Pushed to Kubernetes Cluster

The application container is deployed to the Kubernetes cluster, where the deployment status is monitored.



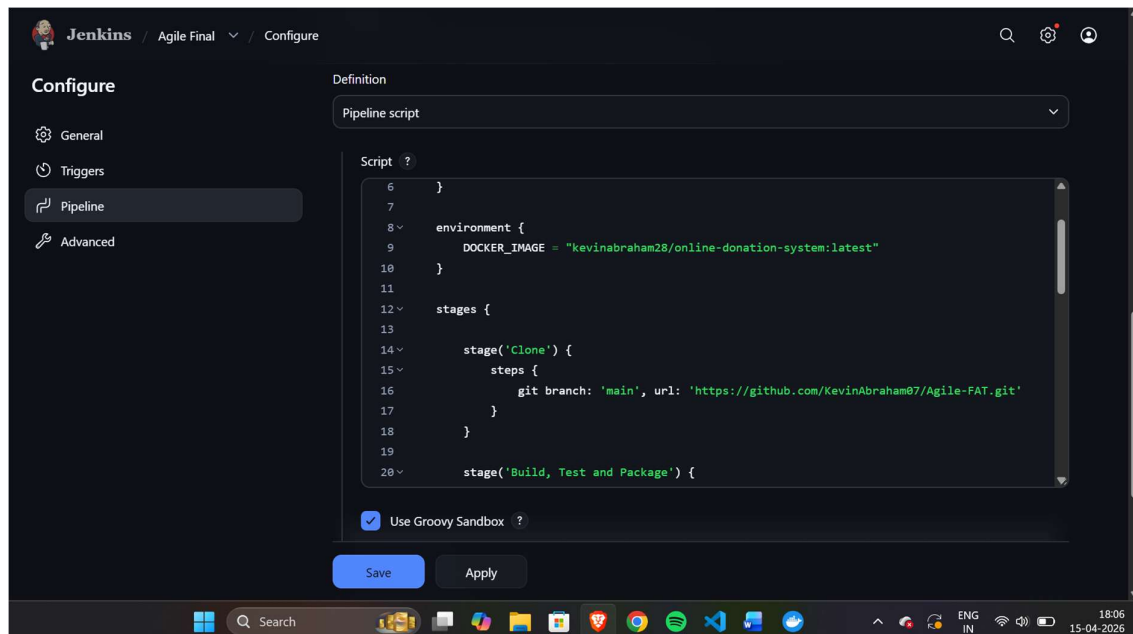
Push to GitHub Repository

The files are uploaded to GitHub to facilitate cloning in Jenkins



Jenkins Pipeline

The Jenkins pipeline configuration automates the CI/CD workflow by cloning the repository, building the project, running tests, containerizing the application, pushing the image, and deploying to Kubernetes.



CI/CD Pipeline Stages

The automated CI/CD pipeline is successfully executed

